Verificação e Validação de Software

Trabalho 2: Cobertura de Código e Teste de Mutação da Tarifa de Estacionamento

Eduardo Alcaria
Arthur Pimentel
Ethan Soares

Introdução

Este relatório aplica cobertura de código (JaCoCo 0.8.12) e teste de mutação (PIT 1.17.4 com `pitest-junit5-plugin` 1.2.1) sobre a classe `CalculadoraTarifa` e a suíte JUnit 5 produzida no Trabalho 1. O ambiente usado foi Java 17 e Maven 3.9. A suíte original possui 25 execuções de teste (`CalculadoraTarifaTest`); os casos novos deste trabalho estão na classe driver `CalculadoraTarifaComplementarTest`, com 27 execuções, totalizando 52 execuções, todas aprovadas.

1 Análise crítica da cobertura de código (JaCoCo)

1.1 Resultado inicial

Métrica	Perdidos	Cobertos	Cobertura
Instruções	0	160	100%
Branches	0	20	100%
Linhas	0	38	100%
Métodos	0	9	100%
Complexidade ciclomática	0	19	100%

Cobertura por método (todos com 0 itens perdidos): `calcular` (31 instruções, 2 branches), `calcularPerNoite` (16, 2), `calcularDescontoVip` (8, 2), `calcularPorPermanencia` (31, 4), `contarPernoites` (24, 2), `validarEntrada` (11, 2), `validarSaida` (23, 6), além do construtor e do inicializador estático.

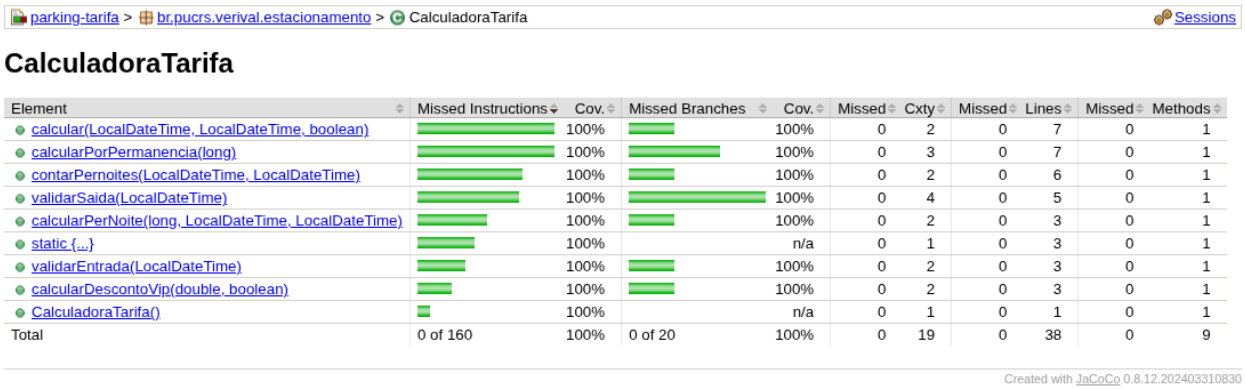


Figura 1: Relatório do JaCoCo para `CalculadoraTarifa`: 0 de 160 instruções e 0 de 20 branches perdidos.

1.2 Análise crítica

A suíte do Trabalho 1 já atinge 100% em todas as métricas do JaCoCo. Portanto **não há linha, branch ou método sem cobertura** e nenhum caso de teste é estritamente necessário para elevar a cobertura. Isso decorre de a suíte ter sido projetada por particionamento em classes de equivalência e análise de valor limite sobre cada regra de negócio, o que exercita naturalmente os dois lados de cada decisão.

Cobertura de 100% não prova que os testes verifiquem o resultado correto, apenas que o código foi executado. Por isso foram acrescentados casos complementares, orientados à técnica de projeto (e não a uma lacuna do JaCoCo), em regiões onde a suíte original era rasa:

- Cliente VIP exercitado apenas em 3 casos; faltavam fronteiras de 60, 61 e 121 minutos e múltiplas diárias.
- Pernoite testado com 1 e 2 diárias; faltava 3 diárias, virada de mês, virada de ano e ano bissexto.
- Janela de saída proibida testada só nos limites 02:00 e 07:59 e no ponto 05:00; faltavam 02:01 e 07:58.
- Permanência longa no mesmo dia testada até 4 h; faltavam 180/181 min e o teto de 23:59.
- Saída às 01:59 de dois dias depois (uma pernoite, não duas).

1.3 Novos casos de teste (resultados válidos)

ID	Entrada	Saída	VIP	Esperado	Técnica
CC01	10/03/2026 08:00	10/03/2026 11:00	não	25,00	Valor limite
CC02	10/03/2026 08:00	10/03/2026 11:01	não	30,00	Valor limite
CC03	10/03/2026 08:00	10/03/2026 23:59	não	90,00	Valor limite
CC04	10/03/2026 22:00	10/03/2026 23:59	não	20,00	Valor limite
CC05	10/03/2026 23:00	11/03/2026 00:00	não	15,00	Valor limite
CC06	10/03/2026 23:59	11/03/2026 01:59	não	20,00	Valor limite
CC07	10/03/2026 23:59	11/03/2026 08:00	não	50,00	Valor limite

ID	Entrada	Saída	VIP	Esperado	Técnica
CC08	10/03/2026 20:00	11/03/2026 08:01	não	50,00	Valor limite
CC09	10/03/2026 20:00	12/03/2026 01:59	não	50,00	Classe equiv.
CC10	10/03/2026 10:00	13/03/2026 10:00	não	150,00	Classe equiv.
CC11	31/03/2026 20:00	01/04/2026 08:00	não	50,00	Valor limite
CC12	31/12/2026 20:00	01/01/2027 08:00	não	50,00	Valor limite
CC13	28/02/2028 20:00	01/03/2028 08:00	não	100,00	Valor limite
CC14	10/03/2026 08:00	10/03/2026 09:00	sim	7,50	Valor limite
CC15	10/03/2026 08:00	10/03/2026 09:01	sim	10,00	Valor limite
CC16	10/03/2026 08:00	10/03/2026 10:01	sim	12,50	Valor limite
CC17	10/03/2026 08:00	10/03/2026 08:20	sim	0,00	Valor limite
CC18	10/03/2026 20:00	12/03/2026 08:00	sim	50,00	Classe equiv.
CC19	10/03/2026 10:00	13/03/2026 10:00	sim	75,00	Classe equiv.

1.4 Novos casos de teste (resultados inválidos)

Todos esperam `IllegalArgumentException`.

ID	Entrada	Saída	VIP	Motivo
CI01	10/03/2026 20:00	11/03/2026 02:01	não	Saída na janela proibida
CI02	10/03/2026 20:00	11/03/2026 07:58	não	Saída na janela proibida
CI03	10/03/2026 20:00	12/03/2026 05:00	não	Saída na janela proibida
CI04	10/03/2026 07:59	10/03/2026 09:00	sim	Entrada antes das 08:00
CI05	10/03/2026 20:00	11/03/2026 03:00	sim	Saída na janela proibida
CI06	10/03/2026 10:00	10/03/2026 09:59	sim	Saída antes da entrada
CI07	10/03/2026 10:00	10/03/2026 10:00	sim	Saída igual à entrada
CI08	10/03/2026 10:00	09/03/2026 12:00	não	Saída em dia anterior

1.5 Resultado após os novos casos

Cobertura inalterada em 100% (160/160 instruções, 20/20 branches, 38/38 linhas, 9/9 métodos), o que é esperado, já que não havia lacuna a fechar.

2 Análise crítica do teste de mutação (PIT)

2.1 Resultado inicial

Com o grupo de mutadores `DEFAULTS`:

Mutantes gerados	32
Mortos	30
Sobreviventes	2
Mutation score	93,75%
Mutation score ajustado (sem equivalentes)	100%

Para uma verificação mais rigorosa, a análise foi repetida com os grupos `STRONGER` (42 mutantes: 39 mortos, 1 *timeout*, 2 sobreviventes) e `ALL` (94 mutantes: 88 mortos, 4 *timeout*, 2 sobreviventes). Os *timeouts* ocorrem em `contarPernoites`: mutantes que tornam o `while` infinito, contabilizados pelo PIT como detectados.

2.2 Mutantes sobreviventes

Os mesmos dois sobreviventes aparecem em todos os grupos, ambos em `calcularPorPermanencia`, linha 53, `if (minutos <= MINUTOS_TARIFA_FIXA)`:

Mutador	Mutação	Classificação
`ConditionalsBoundaryMutator`	`<=` vira `<`	Equivalente
`RemoveConditionalMutator_ORDER_ELSE`	condição substituída por `false`	Equivalente

Mutante 1 (`<=` para `<`). A única entrada em que o comportamento da condição muda é `minutos == 60`. No original retorna `15,0`. No mutante, o fluxo segue: `minutosExcedentes = 0`, `horasAdicionais = (0 + 59) / 60 = 0`, resultado `15 + 0 * 5 = 15,0`. Mesma saída, logo equivalente.

Mutante 2 (condição sempre falsa). Para `21 <= minutos <= 60` (a cortesia já foi tratada na linha 50), `minutosExcedentes` está em $[-39, 0]$, então `minutosExcedentes + 59` está em $[20, 59]$ e a divisão inteira por 60 resulta em 0, gerando `15,0`, idêntico ao original. Logo equivalente.

Como ambos são equivalentes, nenhum teste é capaz de matá-los sem alterar o código de produção. Opcionalmente, o ramo da linha 53 poderia ser removido (refatoração que preserva o comportamento e elimina a redundância), mas isso foge ao escopo do trabalho, que consiste em complementar testes.

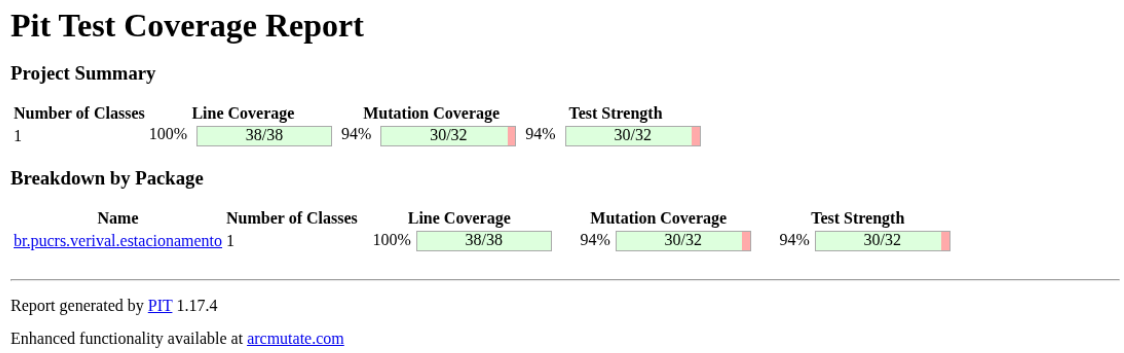


Figura 2: Resumo do PIT (`DEFAULTS`): 30 de 32 mutantes mortos.

```
49      private double calcularPorPermanencia(long minutos) {
50 2        if (minutos <= MINUTOS_CORTESIA) {
51              return 0.0;
52          }
53 2        if (minutos <= MINUTOS_TARIFA_FIXA) {
54 1            return VALOR_TARIFA_FIXA;
55          }
56 1        long minutosExcedentes = minutos - MINUTOS_TARIFA_FIXA;
57 3        long horasAdicionais = (minutosExcedentes + MINUTOS_TARIFA_FIXA - 1) / MINUTOS_TARIFA_FIXA;
58 3        return VALOR_TARIFA_FIXA + horasAdicionais * VALOR_HORA_ADICIONAL;
59      }
```

Figura 3: Código anotado pelo PIT: a linha 53 (em vermelho) concentra os dois mutantes sobreviventes.

2.3 Análise crítica

A suíte do Trabalho 1 já mata todos os mutantes não equivalentes, inclusive com mutadores além do padrão. Não há mutante não equivalente a cobrir com novos casos. Os casos CC01 a CC19 e CI01 a CI08 reforçam as regiões de fronteira descritas na Seção 1, reduzindo o risco de regressões não capturadas pelo conjunto de mutadores do PIT (por exemplo, falhas de arredondamento em horas adicionais e virada de calendário).

Para medir a contribuição própria dos casos novos, o PIT foi executado com o grupo `ALL` usando *apenas* a classe `CalculadoraTarifaComplementarTest`: dos 94 mutantes, 88 foram mortos, 3 terminaram em *timeout* e 3 sobreviveram. Os sobreviventes são os dois equivalentes da linha 53 e o

mutante `InlineConstantMutator` da linha 50 (`20` substituído por `21`), que os casos novos não distinguem porque nenhum deles tem exatamente 21 minutos sem VIP. Esse mutante é morto pela suíte do Trabalho 1 (caso CT03, 21 min). Somadas as duas classes, restam apenas os dois equivalentes.

2.4 Novos casos de teste e mutantes alvo

A tabela relaciona cada grupo de casos novos ao método e ao tipo de mutação que ele foi projetado para exercitar. A última coluna vem da execução do PIT com `ALL` e somente a classe complementar, e conta os mutantes de cada método mortos por *todos* os casos da classe, não por caso individual.

Casos	Método alvo	Mutações exercitadas	Mutantes mortos (classe nova)
CC01 a CC06	`calcularPorPermanencia`	fronteira de `<=`, constantes 20/60, cálculo de horas adicionais (aritmética e arredondamento)	23 de 26
CC07 a CC13	`contarPernoites`, `calcularPerNoite`	fronteira do laço `while`, avanço de `plusDays`, multiplicação pelo valor da diária	13 mortos e 3 *timeout* de 16; 12 de 12
CC14 a CC19	`calcularDescontoVip`	negação do `if (vip)`, troca de `*` por `/`, valor de retorno	7 de 7
CI01 a CI03, CI05	`validarSaida`	limites da janela 02:00–07:59, negações e remoção de condicionais	15 de 15
CI04	`validarEntrada`	limite de `isBefore(ABERTURA)`	6 de 6
CI06 a CI08	`calcular`	negação de `!saida.isAfter(entrada)`, remoção das chamadas de validação	12 de 12

Os casos individuais, com entradas e saídas esperadas, estão nas tabelas da Seção 1. Nenhum deles mata os dois mutantes equivalentes da linha 53, pois eles não alteram o comportamento observável.

2.5 Resultado após os novos casos

Métrica	Antes	Depois
Execuções de teste	25	52
Mutantes gerados (`DEFAULTS`)	32	32
Mortos (`DEFAULTS`)	30	30
Sobreviventes (`DEFAULTS`)	2	2
Mutation score (`DEFAULTS`)	93,75%	93,75%
Mutantes gerados (`ALL`)	94	94
Sobreviventes (`ALL`)	2	2

O score não muda porque os únicos sobreviventes são equivalentes. Os novos casos ampliam a verificação do comportamento sem alterar o resultado das métricas.

3 Conclusão: aceitação da classe

O objetivo do trabalho é decidir, a partir da cobertura e da mutação, se a classe entregue pode ser aceita ou deve ser rejeitada. Os critérios adotados e os resultados medidos são:

Critério de aceitação	Resultado	Atende
Cobertura de instruções e branches de 100%	160/160 e 20/20	sim
Cobertura de linhas e métodos de 100%	38/38 e 9/9	sim
Todos os mutantes não equivalentes mortos (`DEFAULTS`, `STRONGER`, `ALL`)	0 sobreviventes não equivalentes	sim
Sobreviventes restantes justificados	2, ambos equivalentes (prova na Seção 2)	sim
Todos os testes aprovados	52 de 52	sim

Veredito: a classe `CalculadoraTarifa` é aceita. A suíte executa todo o código, exercita os dois lados de cada decisão e detecta toda alteração de comportamento introduzida pelos mutadores do PIT, exceto duas que não mudam o comportamento observável. A aceitação decorre da qualidade da suíte do Trabalho 1, já suficiente; os casos novos reforçam a verificação, mas não eram necessários para atingir os critérios.

Ressalva: cobertura e mutação medem a qualidade dos testes frente ao código existente, e não provam ausência de defeitos. Um requisito mal interpretado e testado de forma coerente com essa interpretação não seria detectado por essas técnicas.

4 Como reproduzir

```
mvn test
mvn test-compile org.pitest:pitest-maven:mutationCoverage
mvn test-compile org.pitest:pitest-maven:mutationCoverage -Dmutators=ALL
```

- Relatório JaCoCo: `target/site/jacoco/index.html`
- Relatório PIT: `target/pit-reports/index.html`